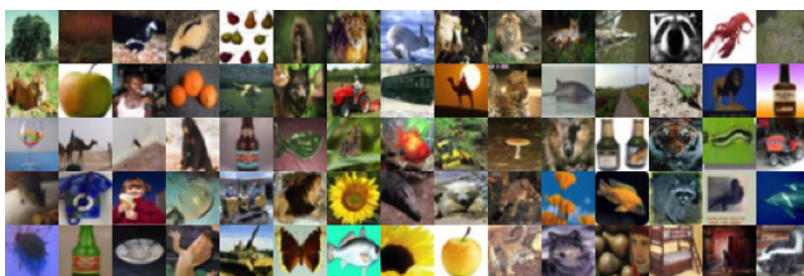


10.3 Convolutional Neural Networks

Neural networks trở lại mạnh mẽ vào khoảng năm 2010 nhờ những thành công lớn trong image classification. Cũng vào thời điểm đó, các databases khổng lồ gồm labeled images với số lượng classes ngày càng tăng được tích lũy. Figure 10.5 cho thấy 75 images lấy từ database CIFAR100.⁴ Database này gồm 60,000 images được gán nhãn theo 20 superclasses (ví dụ aquatic mammals), với năm classes trong mỗi superclass (beaver, dolphin, otter, seal, whale). Mỗi image có resolution 32×32 pixels, với ba eight-bit numbers ở mỗi pixel biểu diễn red, green và blue. Các numbers của mỗi image được tổ chức thành một three-dimensional array gọi là *feature map*. Hai axes đầu tiên là spatial (đều có dimension 32), còn axis thứ ba là *channel axis*,⁵ biểu diễn ba colors. Có một designated training set gồm 50,000 images và một test set gồm 10,000 images.

feature map;
channel



HÌNH 10.5. Một sample images từ database CIFAR100: collection các natural images từ đời sống thường ngày, đại diện cho 100 classes khác nhau.

Một special family của *convolutional neural networks* (CNNs) đã được phát triển để classify các images kiểu này và đạt thành công nổi bật trên rất nhiều problems. Ở một mức độ nào đó, CNNs mô phỏng cách con người classify images: nhận diện những features hoặc patterns cụ thể xuất hiện ở bất kỳ vị trí nào trong image và phân biệt object classes. Trong section này, ta overview ngắn gọn cách chúng hoạt động.

convolutional
neural networks

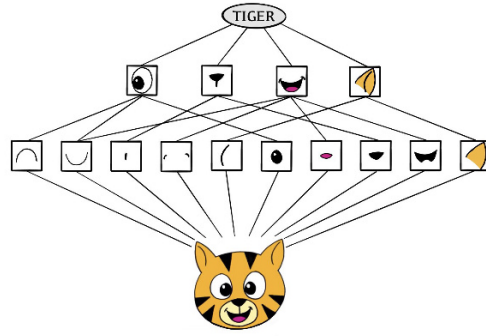
Figure 10.6 minh họa idea của một convolutional neural network trên cartoon image của tiger.⁶ Network trước hết nhận diện low-level features trong input image, như small edges, patches of color, v.v. Những low-level features này sau đó được combine thành higher-level features, chẳng hạn parts of ears, eyes, v.v. Cuối cùng, sự presence hoặc absence của các higher-level features này đóng góp vào probability của từng output class.

Convolutional neural network xây hierarchy này bằng cách combine hai loại specialized hidden layers: *convolution layers* và *pooling layers*. Convolution layers tìm các instances của small patterns trong image, còn pooling layers downsample chúng để chọn một prominent subset. Để đạt state-of-the-art results, các contemporary neural-network architectures dùng nhiều

⁴Xem Chapter 3 của Krizhevsky (2009), “Learning multiple layers of features from tiny images”, tại <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.

⁵Thuật ngữ *channel* được lấy từ signal-processing literature. Mỗi channel là một source of information riêng biệt.

⁶Cảm ơn Elena Tuzhilina đã tạo diagram và <https://www.cartooning4kids.com/> đã cho phép sử dụng cartoon tiger.



HÌNH 10.6. Schematic minh họa cách một convolutional neural network classify image của một con tiger. Network nhận image làm input và nhận diện local features. Sau đó nó combine các local features để tạo compound features, trong example này gồm eyes và ears. Các compound features được dùng để output label “tiger”.

convolution và pooling layers. Tiếp theo ta mô tả hai loại layer này.

10.3.1 Convolution Layers

Một convolution layer gồm nhiều *convolution filters*; mỗi filter là một template dùng để xác định một particular local feature có hiện diện trong image hay không. Convolution filter dựa trên một operation rất đơn giản, gọi là *convolution*, về cơ bản là repeatedly multiply matrix elements rồi add results.

Để hiểu convolution filter hoạt động thế nào, xét một image 4×3 rất đơn giản:

$$\text{Original Image} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}.$$

convolution layer;
convolution filter

Bây giờ xét một filter 2×2 :

$$\text{Convolution Filter} = \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}.$$

Khi convolve image với filter, ta thu được⁷

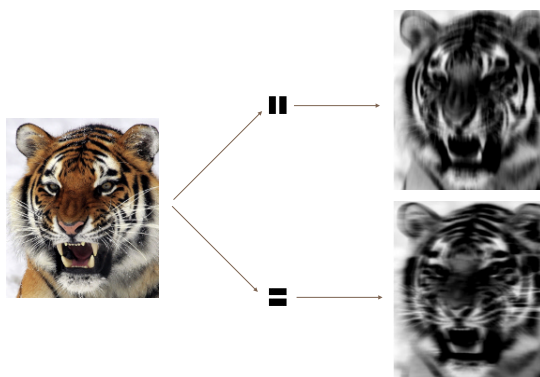
$$\text{Convolved Image} = \begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}.$$

Chẳng hạn, top-left element được tạo bằng cách multiply từng element trong 2×2 filter với corresponding element trong top-left 2×2 portion của image rồi add results. Các elements khác được tạo tương tự: convolution filter được apply lên mọi 2×2 submatrix của original image để tạo convolved

⁷Convolved image nhỏ hơn original image vì dimension của nó bằng number of 2×2 submatrices trong original image. 2×2 ở đây chính là dimension của convolution filter. Nếu muốn convolved image có cùng dimension với original image, có thể dùng *padding*.

image. Nếu một 2×2 submatrix của original image resemble convolution filter thì nó sẽ nhận large value trong convolved image; nếu không, value sẽ nhỏ. Vì vậy convolved image highlight các regions của original image resemble filter. Ta dùng 2×2 chỉ làm example; nói chung convolution filters là các small $\ell_1 \times \ell_2$ arrays, với ℓ_1 và ℓ_2 là small positive integers và không nhất thiết bằng nhau.

Figure 10.7 minh họa việc apply hai convolution filters lên image tiger kích thước 192×179 .⁸ Mỗi convolution filter là image 15×15 chủ yếu gồm zeros (black), với một narrow strip of ones (white) oriented theo vertical hoặc horizontal direction. Khi mỗi filter được convolve với tiger image, các areas resemble filter (có horizontal/vertical stripes hoặc edges) nhận large values, còn areas không resemble feature nhận small values. Convolved images được hiển thị ở bên phải. Horizontal-stripe filter picks out horizontal stripes và edges, còn vertical-stripe filter picks out vertical stripes và edges.



HÌNH 10.7. Convolution filters tìm local features trong image, như edges và small shapes. Bắt đầu với tiger image bên trái và apply hai small convolution filters ở giữa. Convolved images highlight những vùng trong original image có details tương tự filters. Cụ thể, top convolved image highlight vertical stripes của tiger, còn bottom convolved image highlight horizontal stripes. Có thể xem original image như input layer trong convolutional neural network, còn convolved images là units trong first hidden layer.

Figure 10.7 dùng một large image và hai large filters chỉ để illustration. Với CIFAR100, mỗi image có 32×32 color pixels và ta dùng 3×3 convolution filters.

Trong một convolution layer, ta dùng cả một bank of filters để pick out nhiều edges và shapes với orientations khác nhau trong image. Dùng predefined filters theo cách này là standard practice trong image processing. Ngược lại, với CNNs, filters được learned cho specific classification task. Có thể xem filter weights như parameters đi từ input layer tới hidden layer, với một hidden unit cho mỗi pixel trong convolved image. Thực tế đúng là như vậy, nhưng parameters có structure và constraints rất mạnh (xem Exercise 4). Chúng operate trên localized patches của input image (nên có nhiều structural zeros), và cùng weights trong một filter được reuse cho tất

⁸Tiger image dùng trong Figures 10.7–10.9 lấy từ public-domain image resource <https://www.needpix.com/>.

cả patches có thể có trong image (nên weights bị constrained).⁹

Một số details bổ sung:

- Vì input image là color, nó có three channels biểu diễn bởi một three-dimensional feature map. Mỗi channel là một two-dimensional 32×32 feature map—một cho red, một cho green, một cho blue. Một convolution filter cũng có three channels, một cho mỗi color, mỗi channel dimension 3×3 , có thể có filter weights khác nhau. Results của three convolutions được summed để tạo một two-dimensional output feature map. Tại point này, color information đã được dùng và không pass trực tiếp tới các subsequent layers, ngoài vai trò của nó trong convolution.
- Nếu dùng K convolution filters khác nhau ở first hidden layer, ta thu được K two-dimensional output feature maps; cùng nhau chúng được xem như một single three-dimensional feature map. Mỗi output feature map là một separate channel of information, nên bây giờ có K channels thay vì three color channels của original input feature map. Three-dimensional feature map này giống activations trong hidden layer của simple neural network, nhưng được organized và produced theo spatial structure.
- Thông thường ta apply ReLU activation function (10.5) lên convolved image. Bước này đôi khi được xem như một separate layer trong CNN và gọi là *detector layer*.

detector layer

10.3.2 Pooling Layers

Một *pooling layer* cung cấp cách condense một large image thành smaller summary image. Có nhiều cách pooling; *max pooling* summarize mỗi non-overlapping 2×2 block of pixels bằng maximum value trong block. Cách này giảm image size đi factor two ở mỗi direction và đồng thời tạo một mức location invariance: miễn là một trong bốn pixels trong block có large value thì cả block sẽ register là large value trong reduced image.

pooling

Một simple example của max pooling:

$$\text{Max pool} \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}.$$

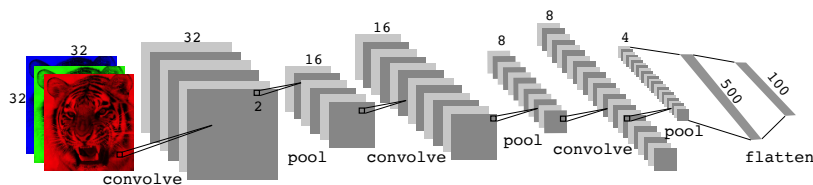
10.3.3 Architecture of a Convolutional Neural Network

Cho đến đây ta đã define một convolution layer: mỗi filter tạo một new two-dimensional feature map. Number of convolution filters trong một convolution layer analogous to number of units ở một hidden layer của fully-connected neural network như Section 10.2. Number này cũng define number of channels trong resulting three-dimensional feature map. Ta cũng đã mô tả pooling layer, làm giảm first two dimensions của mỗi three-

⁹Trong những năm đầu của neural networks, điều này từng được gọi là *weight sharing*.

dimensional feature map. Deep CNNs có nhiều layers kiểu này. Figure 10.8 cho một typical architecture cho CNN trong CIFAR100 image classification task.

Ở input layer, ta thấy three-dimensional feature map của color image, trong đó channel axis biểu diễn mỗi color bằng một 32×32 two-dimensional feature map of pixels. Mỗi convolution filter tạo một new channel ở first hidden layer, mỗi channel là 32×32 feature map (sau khi padding ở edges). Sau first round of convolutions, ta có một new “image”: feature map với nhiều channels hơn hẳn three color input channels (six trong figure, vì ta dùng six convolution filters).



HÌNH 10.8. Architecture của deep CNN cho CIFAR100 classification task. Convolution layers xen kẽ với 2×2 max-pool layers, làm giảm size đi factor 2 theo cả hai dimensions.

Tiếp theo là max-pool layer, làm giảm size của feature map trong mỗi channel đi factor four: factor two ở mỗi dimension. Sequence convolve-then-pool này được repeat cho next two layers. Một số details:

- Mỗi subsequent convolve layer tương tự first one. Nó nhận three-dimensional feature map từ previous layer làm input và treat như một single multi-channel image. Mỗi learned convolution filter có cùng number of channels như feature map này.
- Vì channel feature maps giảm size sau mỗi pool layer, ta thường increase number of filters trong next convolve layer để compensate.
- Đôi khi ta repeat several convolve layers trước một pool layer. Cách này effectively tăng dimension của filter.

Các operations này được repeat cho đến khi pooling giảm mỗi channel feature map xuống chỉ còn vài pixels ở mỗi dimension. Lúc đó three-dimensional feature maps được *flattened*—pixels được treated như separate units—rồi fed vào một hoặc nhiều fully-connected layers trước khi đến output layer, là softmax activation cho 100 classes (như (10.13)).

Có nhiều tuning parameters phải select khi construct network kiểu này, ngoài number, nature và sizes của each layer. Dropout learning có thể dùng ở mỗi layer, cùng lasso hoặc ridge regularization (Section 10.7). Details của việc construct CNN có thể trông daunting; may mắn là software rất tốt đã có sẵn, kèm extensive examples và vignettes hướng dẫn sensible parameter choices. Với official CIFAR100 test set, best accuracy tại thời điểm sách được viết chỉ trên 75%, và performance chắc chắn sẽ còn improve.

10.3.4 Data Augmentation

Một important trick khác trong image modeling là *data augmentation*. Về cơ bản, mỗi training image được replicate nhiều lần, mỗi replicate được randomly distorted theo một cách natural sao cho human recognition không bị ảnh hưởng. Figure 10.9 cho một số examples. Typical distortions gồm zoom, horizontal/vertical shift, shear, small rotations và, trong example này, horizontal flips.

data augmentation



HÌNH 10.9. *Data augmentation. Original image (leftmost) được distort theo các natural ways để tạo different images cùng class label. Các distortions này không đánh lừa humans và hoạt động như một form of regularization khi fit CNN.*

Nhìn trực tiếp, đây là cách enlarge training set đáng kể bằng somewhat different examples, qua đó protect against overfitting. Thực ra có thể xem nó như một form of regularization: ta build một cloud of images quanh mỗi original image, tất cả có cùng label. Kiểu “fattening” data này tương tự về spirit với ridge regularization.

Section 10.7.2 sẽ cho thấy stochastic gradient descent algorithms để fit deep learning models repeatedly process randomly selected batches, ví dụ 128 training images một lần. Data augmentation phối hợp rất tự nhiên với process này vì ta có thể distort each image trong batch on the fly, không cần store tất cả new images.

10.3.5 Results Using a Pretrained Classifier

Ở đây ta dùng một industry-level pretrained classifier để predict class của một số new images. Classifier **resnet50** là một convolutional neural network được trained bằng dataset **imagenet**, gồm millions of images thuộc số lượng categories ngày càng tăng.¹⁰ Figure 10.10 demonstrate performance của **resnet50** trên six photographs (private collection của một author).¹¹ CNN classify hawk trong second image khá tốt. Nếu zoom out như third image, nó bị confused và chọn fountain thay vì hawk. Trong final image, “jacamar” là tropical bird từ South và Central America có coloring tương tự South African Cape Weaver. Section 10.9.4 trình bày thêm details.

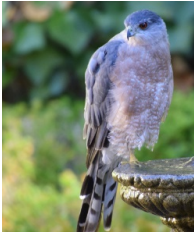
Phần lớn work khi fit CNN là learn convolution filters ở hidden layers; đây là coefficients của CNN. Với models fit trên massive corpora như **imagenet** có nhiều classes, output của các filters này có thể serve as features cho general natural-image classification problems. Ta có thể reuse các

¹⁰Thông tin thêm về **resnet50**: He, Zhang, Ren, and Sun (2015), “Deep residual learning for image recognition”, <https://arxiv.org/abs/1512.03385>. Chi tiết về **imagenet**: Russakovsky, Deng, et al. (2015), “ImageNet Large Scale Visual Recognition Challenge”, *International Journal of Computer Vision*.

¹¹Các **resnet** results này có thể thay đổi theo thời gian vì publicly-trained model được update định kỳ.

pretrained hidden layers cho new problems với much smaller training sets (process gọi là *weight freezing*), rồi chỉ train last few layers của network, nên cần much less data.

weight freezing

		
flamingo	Cooper's hawk	Cooper's hawk
flamingo 0.83	kite 0.60	fountain 0.35
spoonbill 0.17	great grey owl 0.09	nail 0.12
white stork 0.00	robin 0.06	hook 0.07
Lhasa Apso	cat	Cape weaver
Tibetan terrier 0.56	Old English sheepdog 0.82	jacamar 0.28
Lhasa 0.32	Shih-Tzu 0.04	macaw 0.12
cocker spaniel 0.03	Persian cat 0.04	robin 0.12

HÌNH 10.10. Classification của six photographs bằng *resnet50 CNN* trained trên *imagenet* corpus. Table dưới images hiển thị true (intended) label ở top của mỗi panel và top three choices của classifier (trong 100 classes). Numbers là estimated probabilities cho mỗi choice. (A kite is a raptor, but not a hawk.)

Các vignettes và book¹² đi kèm package `keras` cung cấp thêm details về những applications như vậy.

10.4 Document Classification

Trong section này ta giới thiệu một loại example mới có important applications trong industry và science: predicting attributes of documents. Documents có thể là articles trong medical journals, Reuters news feeds, emails, tweets, v.v. Example của ta là IMDb (Internet Movie Database) ratings—short documents trong đó viewers viết critiques of movies.¹³ Response ở đây là sentiment của review, positive hoặc negative.

Đây là beginning của một negative review khá amusing; vì đoạn này là raw document data nên giữ nguyên tiếng Anh:

This has to be one of the worst films of the 1990s. When my friends & I were watching this film (being the target audience it was aimed at)

¹²*Deep Learning with R* của F. Chollet và J.J. Allaire, 2018, Manning Publications.
¹³Xem Maas et al. (2011), “Learning word vectors for sentiment analysis”, trong *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 142–150.

we just sat & watched the first half an hour with our jaws touching the floor at how bad it really was. The rest of the time, everyone else in the theater just started talking to each other, leaving or generally crying into their popcorn ...

Mỗi review có thể có length khác nhau, chứa slang hoặc non-words, spelling errors, v.v. Ta cần một cách để *featurize* document—modern parlance cho việc define một set of predictors.

featurize

Featurization đơn giản và phổ biến nhất là *bag-of-words model*. Ta score mỗi document theo presence/absence của từng word trong language dictionary—ở đây là English dictionary. Nếu dictionary có M words, thì với mỗi document ta tạo binary feature vector length M , ghi 1 cho mỗi word present và 0 nếu absent. Feature vector này có thể rất wide, nên ta giới hạn dictionary ở 10,000 words xuất hiện frequent nhất trong training corpus gồm 25,000 reviews. Có tools tiện lợi để làm việc này automatically.

bag-of-words

Đây là beginning của một positive review sau khi được redacted theo cách đó; tiếp tục giữ nguyên vì đây là data:

<START> this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert <UNK> is an amazing actor and now the same being director <UNK> father came from the same scottish island as myself so i loved ...

Ta thấy nhiều words đã bị omitted và một số unknown words (UNK) được mark. Sau reduction này, binary feature vector có length 10,000, chủ yếu là zeros và chỉ rải rác ones ở positions tương ứng với words có trong document.

Ta có training set và test set, mỗi set 25,000 examples, đều balanced theo sentiment. Resulting training feature matrix X có dimension $25,000 \times 10,000$, nhưng chỉ 1.3% binary entries là non-zero. Ta gọi matrix như vậy là *sparse*, vì hầu hết values giống nhau (zero ở đây); nó có thể được store efficiently dưới *sparse matrix format*.¹⁴ Có nhiều cách account for document length; ở đây ta chỉ score một word là in/out document, nhưng chẳng hạn có thể record relative frequency của words.

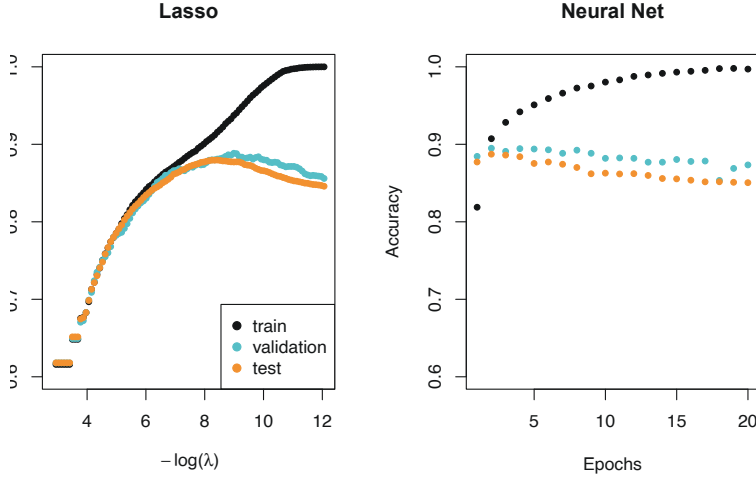
sparse matrix format

Ta tách một validation set size 2,000 khỏi 25,000 training observations (để model tuning), rồi fit two model sequences:

- một lasso logistic regression dùng package `glmnet`;
- một two-class neural network với two hidden layers, mỗi layer có 16 ReLU units.

Cả hai methods tạo ra sequence of solutions. Lasso sequence được indexed bởi regularization parameter λ . Neural-net sequence được indexed bởi number of gradient-descent iterations dùng trong fitting, measured bằng training epochs hay passes through training set (Section 10.7). Training accuracy trong Figure 10.11 (black points) tăng monotonically trong cả hai cases. Ta có thể dùng validation error để pick một good solution từ mỗi sequence (blue points), rồi dùng solution đó để make predictions trên test data.

¹⁴Thay vì store toàn bộ matrix, ta có thể chỉ store locations và values của nonzero entries. Trong case này, vì all nonzero entries đều bằng 1 nên chỉ locations cần được store.



HÌNH 10.11. Accuracy của lasso và two-hidden-layer neural network trên IMDB data. Với lasso, x-axis hiển thị $-\log(\lambda)$; với neural network, x-axis hiển thị epochs (number of times fitting algorithm passes through training set). Cả hai đều có tendency to overfit và đạt approximately same test accuracy.

Một two-class neural network tương đương một nonlinear logistic regression model. Từ (10.12) và (10.13),

$$\begin{aligned} \log \left\{ \frac{\Pr(Y = 1 | X)}{\Pr(Y = 0 | X)} \right\} &= Z_1 - Z_0 \\ &= (\beta_{10} - \beta_{00}) + \sum_{\ell=1}^{K_2} (\beta_{1\ell} - \beta_{0\ell}) A_{\ell}^{(2)}. \end{aligned} \quad (10.15)$$

Điều này cho thấy redundancy trong softmax function: với K classes ta thực sự chỉ cần estimate $K - 1$ sets of coefficients (xem Section 4.3.5). Trong Figure 10.11 ta show accuracy (fraction correct) thay vì classification error (fraction incorrect), vì former popular hơn trong machine learning community. Cả hai models đạt test-set accuracy khoảng 88%.

Bag-of-words model summarize document bằng words present và ignore context. Có ít nhất two popular ways để account for context:

- *Bag-of-n-grams model.* Ví dụ, bag of 2-grams record consecutive co-occurrence của mọi distinct pair of words. “Blissfully long” có thể là positive phrase trong movie review, còn “blissfully short” là negative.
- Treat document như một sequence, account for all words trong context của những words đi trước và theo sau.

Trong next section ta explore models cho sequences of data, có applications trong weather forecasting, speech recognition, language translation và time-series prediction, cùng nhiều lĩnh vực khác. Ta cũng tiếp tục IMDB example ở đó.

10.5 Recurrent Neural Networks

Nhiều data sources có bản chất sequential và cần special treatment khi build predictive models. Examples gồm:

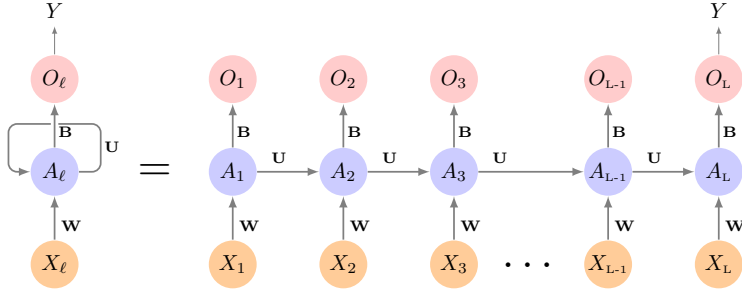
- Documents như book/movie reviews, newspaper articles và tweets. Sequence và relative positions của words trong document capture narrative, theme và tone, và có thể khai thác trong topic classification, sentiment analysis và language translation.
- Time series của temperature, rainfall, wind speed, air quality, v.v. Ta có thể muốn forecast weather vài ngày ahead hoặc climate vài decades ahead.
- Financial time series, nơi ta track market indices, trading volumes, stock/bond prices và exchange rates. Prediction thường khó, nhưng như sẽ thấy, một số indices có thể được predict với reasonable accuracy.
- Recorded speech, musical recordings và other sound recordings. Ta có thể muốn tạo text transcription của speech, hoặc language translation; cũng có thể assess quality của music hay assign attributes.
- Handwriting, như doctor's notes và handwritten digits như zip codes. Mục tiêu là turn handwriting into digital text hoặc read digits (optical character recognition).

Trong một *recurrent neural network* (RNN), input object X là một sequence. Xét corpus of documents như collection IMDb movie reviews. Mỗi document có thể represent như sequence gồm L words, $X = \{X_1, X_2, \dots, X_L\}$, trong đó mỗi X_ℓ represents một word. Order của words và closeness của certain words trong sentence convey semantic meaning. RNNs được design để accommodate và exploit sequential nature của input objects, tương tự cách CNNs exploit spatial structure của image inputs. Output Y cũng có thể là sequence (như language translation), nhưng thường là scalar, ví dụ binary sentiment label của movie review.

recurrent neural network

Figure 10.12 minh họa structure của very basic RNN với input sequence $X = \{X_1, X_2, \dots, X_L\}$, simple output Y và hidden-layer sequence $\{A_\ell\}_1^L = \{A_1, A_2, \dots, A_L\}$. Mỗi X_ℓ là vector; trong document example, X_ℓ có thể represent one-hot encoding cho ℓ th word dựa trên language dictionary của corpus (xem top panel Figure 10.13). Khi sequence được process từng vector X_ℓ , network update activations A_ℓ trong hidden layer, nhận input là X_ℓ và activation vector $A_{\ell-1}$ từ previous step. Mỗi A_ℓ feeds vào output layer và tạo prediction O_ℓ cho Y ; O_L , prediction cuối, là most relevant.

Cụ thể, giả sử mỗi vector X_ℓ trong input sequence có p components $X_\ell^T = (X_{\ell 1}, X_{\ell 2}, \dots, X_{\ell p})$, và hidden layer có K units $A_\ell^T = (A_{\ell 1}, A_{\ell 2}, \dots, A_{\ell K})$. Như Figure 10.4, collection gồm $K \times (p + 1)$ shared weights w_{kj} cho input layer được represented bởi matrix W ; tương tự U là $K \times K$ matrix của weights u_{ks} cho hidden-to-hidden layers, và B là $K + 1$ vector của weights



HÌNH 10.12. Schematic của một simple recurrent neural network. Input là sequence of vectors $\{X_\ell\}_1^L$, còn target ở đây là single response. Network process input sequence X sequentially; mỗi X_ℓ feeds vào hidden layer, layer này cũng nhận activation vector $A_{\ell-1}$ từ previous element và tạo current activation vector A_ℓ . Cùng collections of weights W , U và B được dùng khi process từng element. Output layer tạo sequence predictions O_ℓ từ current activation A_ℓ , nhưng thường chỉ final one, O_L , là relevant. Bên trái dấu bằng là concise representation của network; bên phải là version được unroll rõ hơn.

β_k cho output layer. Khi đó

$$A_{\ell k} = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_{\ell j} + \sum_{s=1}^K u_{ks} A_{\ell-1, s} \right), \quad (10.16)$$

và output O_ℓ được compute bởi

$$O_\ell = \beta_0 + \sum_{k=1}^K \beta_k A_{\ell k}, \quad (10.17)$$

cho quantitative response, hoặc thêm sigmoid activation function với binary response chẳng hạn. Ở đây $g(\cdot)$ là activation function như ReLU.

Lưu ý cùng weights W , U và B được dùng khi process mọi element trong sequence; tức chúng không depend on ℓ . Đây là một form of *weight sharing* trong RNNs, tương tự việc reuse filters trong convolutional neural networks (Section 10.3.1). Khi đi từ beginning tới end, activations A_ℓ accumulate history của những gì đã thấy, để learned context có thể dùng cho prediction.

weight sharing

Với regression problems, loss function cho observation (X, Y) là

$$(Y - O_L)^2, \quad (10.18)$$

chỉ reference final output $O_L = \beta_0 + \sum_{k=1}^K \beta_k A_{Lk}$; vì vậy $O_1, O_2, \dots, O_{L-1}$ không được dùng. Khi fit model, mỗi element X_ℓ của input sequence X contributes tới O_L qua chain (10.16), nên indirectly contributes vào learning shared parameters W , U và B thông qua loss (10.18). Với n input

sequence/response pairs (x_i, y_i) , parameters được tìm bằng cách minimize

$$\sum_{i=1}^n (y_i - o_{iL})^2 = \sum_{i=1}^n \left[y_i - \left\{ \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{iLj} + \sum_{s=1}^K u_{ks} a_{i,L-1,s} \right) \right\} \right]^2 \quad (10.19)$$

Ở đây lowercase letters dùng cho observed y_i và vector sequences $x_i = \{x_{i1}, x_{i2}, \dots, x_{iL}\}$,¹⁵ cũng như derived activations.

Vì intermediate outputs O_ℓ không được dùng, ta có thể hỏi tại sao chúng tồn tại. Trước hết, chúng gần như “free”, vì dùng same output weights B cần để produce O_L , đồng thời provide evolving prediction cho output. Hơn nữa, với một số learning tasks response cũng là sequence, nên output sequence $\{O_1, O_2, \dots, O_L\}$ thực sự cần thiết.

Khi dùng ở full strength, RNNs có thể khá complex. Ta minh họa bằng two simple applications. First, tiếp tục IMDb sentiment analysis từ previous section bằng cách process words trong reviews sequentially. Second, minh họa use trong financial time series forecasting problem.

10.5.1 Sequential Models for Document Classification

Ta quay lại classification task với IMDb reviews. Section 10.4 dùng bag-of-words model; ở đây plan là dùng sequence of words trong document để predict label cho toàn document.

Tuy nhiên có dimensionality problem: mỗi word được represented bởi one-hot-encoded vector (dummy variable) với 10,000 elements—một cho mỗi word trong dictionary. Một approach phổ biến là represent mỗi word trong lower-dimensional *embedding space*. Thay vì binary vector với 9,999 zeros và một single one, mỗi word được represented bởi set gồm m real numbers, thường không có số nào bằng zero. m là embedding dimension và có thể chỉ ở low hundreds hoặc nhỏ hơn. Trong case này ta cần matrix E dimension $m \times 10,000$, mỗi column indexed bởi một trong 10,000 words; values trong column là m coordinates của word trong embedding space.

embedding

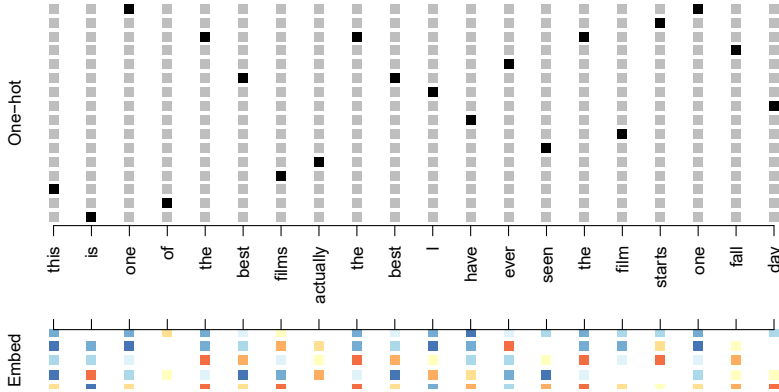
Figure 10.13 minh họa idea này (dictionary 16 thay vì 10,000 words, và $m = 5$). E đến từ đâu? Nếu có large corpus of labeled documents, neural network có thể learn E như một part of optimization. Khi đó E được gọi là *embedding layer*, và specialized E được learned cho task đang xét. Nếu không, ta có thể insert một precomputed matrix E vào embedding layer, process gọi là weight freezing. Hai pretrained embeddings được dùng rộng rãi là *word2vec* và *GloVe*.¹⁶ Chúng được built từ very large corpus of documents bằng variant of principal components analysis (Section 12.2). Idea là positions của words trong embedding space preserve semantic meaning; ví dụ synonyms nên nằm gần nhau.

*embedding layer;
weight freezing;
word2vec; GloVe*

Mỗi document giờ được represented như sequence of m -vectors tương ứng sequence of words. Next step là limit mỗi document vào last L words.

¹⁵Đây là một sequence of vectors; mỗi element $x_{i\ell}$ là một p -vector.

¹⁶word2vec được mô tả trong Mikolov, Chen, Corrado, and Dean (2013), tại <https://code.google.com/archive/p/word2vec>. GloVe được mô tả trong Pennington, Socher, and Manning (2014), tại <https://nlp.stanford.edu/projects/glove>.



HÌNH 10.13. *Depiction của sequence 20 words đại diện một single document: one-hot encoded bằng dictionary 16 words (top panel) và embedded trong m -dimensional space với $m = 5$ (bottom panel).*

Documents shorter than L được padded bằng zeros ở front. Vì vậy mỗi document trở thành series gồm L vectors $X = \{X_1, X_2, \dots, X_L\}$, mỗi X_ℓ có m components.

Ta dùng RNN structure trong Figure 10.12. Training corpus gồm n separate series (documents) length L , mỗi series được process sequentially left-to-right. Trong process này, parallel series of hidden activation vectors A_ℓ , $\ell = 1, \dots, L$, được tạo theo (10.16) cho mỗi document. A_ℓ feeds vào output layer tạo evolving prediction O_ℓ . Final value O_L được dùng để predict response: sentiment của review.

Đây là simple RNN với relatively few parameters. Nếu có K hidden units, common weight matrix W có $K \times (m + 1)$ parameters, matrix U có $K \times K$, và B có $2(K + 1)$ parameters cho two-class logistic regression như (10.15). Chúng được reuse khi process sequence $X = \{X_\ell\}_1^L$ left-to-right, tương tự reuse một single convolution filter cho mỗi patch trong image (Section 10.3.1). Nếu embedding layer E được learned, nó add thêm $m \times D$ parameters ($D = 10,000$ ở đây), và đây là cost lớn nhất.

Ta fit RNN vừa mô tả cho IMDb data. Model có embedding matrix E với $m = 32$ (learned during training, không phải precomputed), followed by single recurrent layer với $K = 32$ hidden units. Model được trained với dropout regularization trên 25,000 reviews trong designated training set và chỉ đạt disappointing 76% accuracy trên IMDb test data. Network dùng pretrained GloVe embedding matrix E còn slightly worse.

Để dễ exposition ta chỉ trình bày very simple RNN. Versions phức tạp hơn dùng *long term and short term memory* (LSTM). Hai tracks của hidden-layer activations được maintained, nên khi activation A_ℓ được compute, nó nhận input từ hidden units both further back in time và closer in time—một so-called *LSTM RNN*. Với long sequences, cách này khắc phục problem early signals bị washed out khi propagated qua chain tới final activation vector A_L .

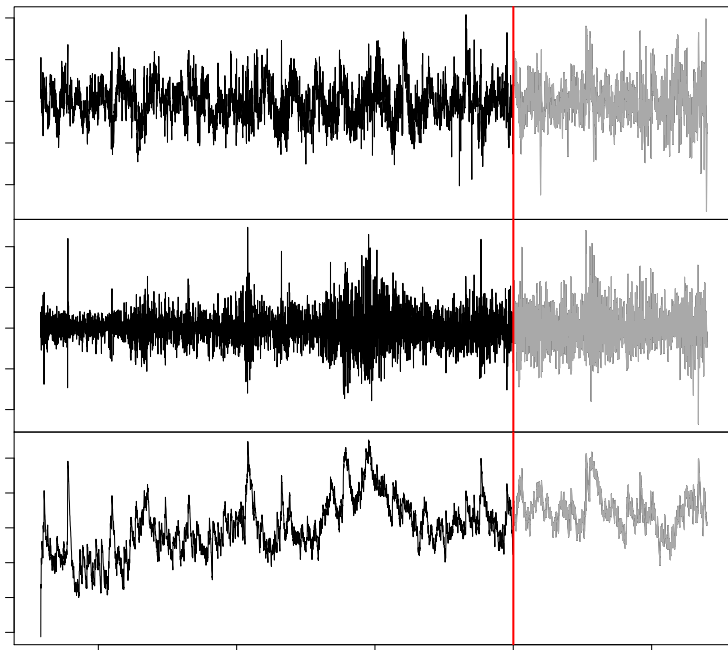
Khi refit model bằng LSTM architecture cho hidden layer, performance improve lên 87% trên IMDb test data, comparable với 88% của bag-of-words model trong Section 10.4. Section 10.9.6 cung cấp fitting details. Dù

thêm LSTM complexity, RNN này vẫn chỉ ở mức “entry level”. Ta có thể improve đôi chút bằng cách change model size, regularization và add hidden layers, nhưng LSTM models train lâu nên exploration nhiều architectures và parameter optimization khá tedious.

RNNs cung cấp rich framework cho sequence modeling và tiếp tục evolve. Đã có nhiều advances về architecture, data augmentation và learning algorithms. Tại thời điểm sách được viết (early 2020), leading RNN configurations report accuracy trên 95% cho IMDB data; details nằm ngoài scope của sách.¹⁷

10.5.2 Time Series Forecasting

Figure 10.14 cho historical trading statistics từ New York Stock Exchange: three daily time series trong period December 3, 1962 đến December 31, 1986.¹⁸



HÌNH 10.14. Historical trading statistics từ New York Stock Exchange. Daily values của normalized log trading volume, DJIA return và log volatility trong 24-year period 1962–1986. Ta muốn predict trading volume của một ngày dựa trên history của all earlier days. Bên trái red bar (January 2, 1980) là training data, bên phải là test data.

Ba series là:

- *Log trading volume*: fraction của all outstanding shares được traded trong ngày, relative to 100-day moving average của past turnover,

¹⁷Một IMDB leaderboard có tại <https://paperswithcode.com/sota/sentiment-analysis-on-imdb>.

¹⁸Data được assembled bởi LeBaron and Weigend (1998), *IEEE Transactions on Neural Networks*, 9(1): 213–220.

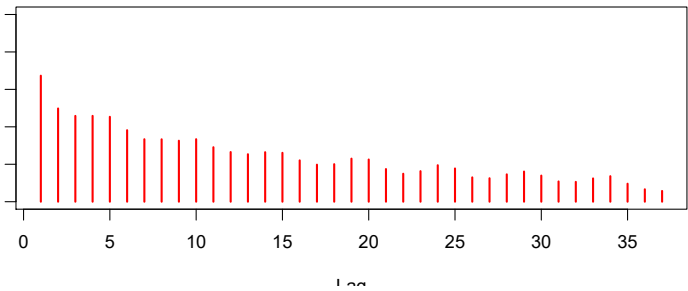
trên log scale.

- *Dow Jones return*: difference giữa log Dow Jones Industrial Index trên consecutive trading days.
- *Log volatility*: based on absolute values của daily price movements.

Predicting stock prices notoriously hard, nhưng predicting trading volume dựa trên recent history lại manageable hơn và useful cho planning trading strategies.

Một observation gồm measurements (v_t, r_t, z_t) ở day t , tương ứng `log_volume`, `DJ_return` và `log_volatility`. Có tổng cộng $T = 6,051$ triples, mỗi series được plot trong Figure 10.14. Một feature nổi bật là day-to-day observations không independent. Series có *autocorrelation*: values gần nhau theo time có tendency similar. Điều này khác các datasets trước, nơi observations thường được assumed independent.

autocorrelation



HÌNH 10.15. Autocorrelation function cho `log_volume`. Nearby values correlated khá strongly, với correlations trên 0.2 tới khoảng lag 20 days.

Cụ thể, xét pairs $(v_t, v_{t-\ell})$ cách nhau lag ℓ days. Lấy all such pairs trong v_t series và compute correlation coefficient cho ta autocorrelation ở lag ℓ . Figure 10.15 show autocorrelation function tới lag 37, với considerable correlation. Một characteristic thú vị khác là response variable v_t —`log_volume`—cũng là predictor: ta dùng past values của `log_volume` để predict future values.

lag

RNN forecaster

Ta muốn predict v_t từ past values $v_{t-1}, v_{t-2}, \dots$ và đồng thời dùng past values của $r_{t-1}, r_{t-2}, \dots$ và $z_{t-1}, z_{t-2}, \dots$. Dù combined data là long series 6,051 trading days, structure của problem khác document-classification example:

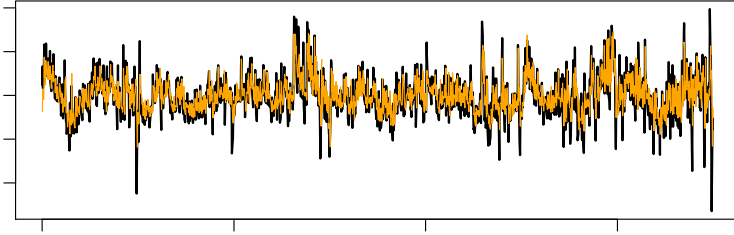
- chỉ có one series of data, không phải 25,000;
- có entire series of targets v_t , và inputs include past values của series này.

Để represent problem theo Figure 10.12, ta extract nhiều short mini-series input sequences $X = \{X_1, X_2, \dots, X_L\}$ với predefined length L (gọi

là lag trong context này) và corresponding target Y :

$$X_1 = \begin{pmatrix} v_{t-L} \\ r_{t-L} \\ z_{t-L} \end{pmatrix}, \quad X_2 = \begin{pmatrix} v_{t-L+1} \\ r_{t-L+1} \\ z_{t-L+1} \end{pmatrix}, \quad \dots, \quad X_L = \begin{pmatrix} v_{t-1} \\ r_{t-1} \\ z_{t-1} \end{pmatrix}, \quad Y = v_t. \quad (10.20)$$

Target Y là `log_volume` value v_t tại single timepoint t ; input sequence X là series of 3-vectors $\{X_\ell\}_1^L$, mỗi vector gồm `log_volume`, `DJ_return`, `log_volatility` từ day $t-L$ đến $t-1$. Mỗi $t = L+1, \dots, T$ tạo một separate (X, Y) pair. Với NYSE data, ta dùng past five trading days để predict next day, nên $L = 5$. Vì $T = 6,051$, ta tạo được 6,046 pairs. Rõ ràng L là parameter cần chọn cẩn thận, có thể dùng validation data.



HÌNH 10.16. RNN forecast của `log_volume` trên NYSE test data. Black lines là true volumes, orange lines superimposed là forecasts. Forecasted series account for 42% variance của `log_volume`.

Ta fit model với $K = 12$ hidden units bằng 4,281 training sequences derived từ data trước January 2, 1980 (Figure 10.14), rồi forecast 1,770 values của `log_volume` sau date này. Test $R^2 = 0.42$. Details ở Section 10.9.6. Là một *straw man*,¹⁹ dùng yesterday's `log_volume` để predict today chỉ đạt $R^2 = 0.18$. Figure 10.16 show forecast results; observed daily values cho 1980–1986 bằng black, predictions bằng orange, và correspondence khá tốt. Khi forecast test period, ta phải dùng chính past test data để form input sequences X ; điều này không phải cheating vì ta luôn dùng past data để predict future.

Autoregression

RNN vừa fit có nhiều điểm chung với traditional *autoregression* (AR) linear model. Trước hết xét response sequence v_t một mình và construct response vector y cùng predictor matrix M :

$$y = \begin{bmatrix} v_{L+1} \\ v_{L+2} \\ v_{L+3} \\ \vdots \\ v_T \end{bmatrix}, \quad M = \begin{bmatrix} 1 & v_L & v_{L-1} & \cdots & v_1 \\ 1 & v_{L+1} & v_L & \cdots & v_2 \\ 1 & v_{L+2} & v_{L+1} & \cdots & v_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{T-1} & v_{T-2} & \cdots & v_{T-L} \end{bmatrix}. \quad (10.21)$$

M và y đều có $T-L$ rows, one per observation. Predictors cho response v_t ở day t là previous L values của same series.

autoregression

¹⁹Straw man ở đây là simple và sensible prediction dùng làm baseline comparison.

Regressing y on M tương đương fit

$$\hat{v}_t = \hat{\beta}_0 + \hat{\beta}_1 v_{t-1} + \hat{\beta}_2 v_{t-2} + \cdots + \hat{\beta}_L v_{t-L}, \quad (10.22)$$

gọi là order- L autoregressive model, hay $AR(L)$. Với NYSE data ta có thể include lagged versions của `DJ_return` và `log_volatility`, r_t và z_t , trong predictor matrix M , tạo $3L + 1$ columns. AR model với $L = 5$ đạt test $R^2 = 0.41$, slightly inferior to RNN's 0.42.

RNN và AR models rất similar: cùng response Y và input sequences X length $L = 5$, dimension $p = 3$. RNN process sequence left-to-right với same weights W (input layer), trong khi AR model simply treat all L sequence elements equally như vector of $L \times p$ predictors—process gọi là *flattening*. RNN còn có hidden activations A_ℓ truyền information dọc sequence và add nonlinearity. Từ (10.19), với $K = 12$ hidden units, RNN có $13 + 12(1 + 3 + 12) = 205$ parameters, so với 16 của $AR(5)$.

flattening

Một obvious extension của AR model là dùng set of lagged predictors làm input vector cho ordinary feedforward neural network (10.1), thêm flexibility. Cách này đạt test $R^2 = 0.42$, slightly better linear AR và same as RNN.

Tất cả models có thể improve nếu include variable `day_of_week` tương ứng day t của target v_t (learn được từ calendar dates); trading volume thường higher vào Mondays và Fridays. Với five trading days, variable này one-hot encode thành five binary variables. AR và RNN đều improve lên $R^2 = 0.46$, còn nonlinear AR lên $R^2 = 0.47$.

Ta dùng simplest RNN version trong examples. Experiments bổ sung với LSTM extension chỉ yield small improvements, thường tới khoảng 1% trong R^2 . Fitting details của cả three models ở Section 10.9.6.

10.5.3 Summary of RNNs

Ta đã illustrate RNNs bằng two simple use cases và mới chỉ scratched the surface. Có nhiều variations và enhancements. Một approach không discuss ở trên dùng one-dimensional convolutional neural network, treat sequence of vectors (ví dụ words represented trong embedding space) như image. Convolution filter slides dọc sequence theo one dimension, có thể learn particular phrases hoặc short subsequences relevant cho task.

RNN cũng có thể có additional hidden layers. Với two hidden layers chẳng hạn, sequence A_ℓ được treat như input sequence cho next hidden layer theo natural way.

RNN trong example scan document beginning-to-end; *bidirectional RNNs* thay vào đó scan sequences theo both directions. Trong language translation, target cũng là sequence of words ở một language khác input. Cả input sequence và target sequence được represented bằng structure tương tự Figure 10.12 và share hidden units. Trong so-called *Seq2Seq learning*, hidden units được xem như capture semantic meaning của sentences. Một số breakthroughs lớn trong language modeling và translation đến từ recent improvements của RNNs.

*bidirectional;
Seq2Seq*

Algorithms để fit RNNs có thể complex và computationally costly. Fortunately, good software che bớt complexities và làm specification/fitting khá painless. Nhiều models ta dùng hằng ngày (như Google Translate) dùng state-of-the-art architectures phát triển bởi teams of highly skilled

engineers và được trained bằng massive computational và data resources.

10.6 When to Use Deep Learning

Performance của deep learning trong chapter này rất impressive: digit classification được xử lý rất tốt và deep CNNs đã revolutionize image classification. Ta thường xuyên thấy new success stories, nhiều trong số đó liên quan image classification như machine diagnosis of mammograms/digital X-rays, ophthalmology scans, MRI annotations, v.v. RNNs cũng thành công trong speech/language translation, forecasting và document modeling. Câu hỏi tự nhiên là: có nên bỏ các older tools và dùng deep learning cho mọi problem with data? Để trả lời, ta revisit **Hitters** dataset từ Chapter 6.

Đây là regression problem: predict **Salary** của baseball player năm 1987 từ performance statistics năm 1986. Sau khi remove players missing response, còn 263 players và 19 variables. Ta randomly split thành training set 176 players (two thirds) và test set 87 players (one third), rồi dùng three methods:

- linear model fit training data và predict test data; model có 20 parameters;
- same linear model với lasso regularization; tuning parameter selected bằng 10-fold cross-validation trên training data, chọn model có 12 variables với nonzero coefficients;
- neural network với one hidden layer gồm 64 ReLU units; model có 1,345 parameters.²⁰

Model	# Parameters	Mean Abs. Error	Test Set R^2
Linear Regression	20	254.7	0.56
Lasso	12	252.3	0.51
Neural Network	1345	257.4	0.54

BẢNG 10.2. Prediction results trên **Hitters** test data cho linear models fit bằng ordinary least squares và lasso, so với neural network fit bằng stochastic gradient descent với dropout regularization.

Table 10.2 compare results. Performance khá similar cho all three models. Ta report mean absolute error trên test data và test R^2 ; tất cả đều respectable (xem Exercise 5). Ta đã spend khá nhiều time fiddling với configuration parameters của neural network để đạt results này. Có thể nếu spend thêm time và chọn form/amount of regularization vừa đúng, neural network có thể match hoặc outperform linear regression và lasso. Nhưng linear models work well với great ease, và much easier to present/understand hơn neural-network black box. Lasso select 12 trong 19 variables. Trong case như vậy, tốt hơn nên theo *Occam's razor*: khi nhiều methods có roughly equivalent performance, choose simplest.

Occam's razor

Sau thêm exploration với lasso, ta identify even simpler model với four variables. Ta refit linear model với four variables trên training data (so-

²⁰Model được fit bằng stochastic gradient descent với batch size 32 trong 1,000 epochs và 10% dropout regularization. Test-error performance flatten rồi bắt đầu slowly increase sau 1,000 epochs. Fitting details ở Section 10.7.

called *relaxed lasso*) và đạt test mean absolute error 224.8, overall winner. Ta có thể muốn present summary table từ fit này để xem coefficients và p-values, nhưng vì model được selected trên training data nên sẽ có selection bias. Thay vào đó, authors refit model trên test data, vốn không dùng trong selection. Table 10.3 cho results.

	Coefficient	Std. error	t-statistic	p-value
Intercept	-226.67	86.26	-2.63	0.0103
Hits	3.06	1.02	3.00	0.0036
Walks	0.181	2.04	0.09	0.9294
CRuns	0.859	0.12	7.09	< 0.0001
PutOuts	0.465	0.13	3.60	0.0005

BẢNG 10.3. *Least squares coefficient estimates associated với regression của Salary lên four variables selected by lasso trên Hitters data. Model này đạt best performance trên test data với mean absolute error 224.8. Results ở đây được obtained từ regression trên test data, vốn không dùng khi fitting lasso model.*

Ta có nhiều powerful tools: neural networks, random forests/boosting, support vector machines, generalized additive models, v.v., cùng linear models và simple variants. Khi gặp new data modeling/prediction problems, rất tempting luôn chọn trendy new methods. Chúng thường rất impressive, đặc biệt khi datasets extremely large và support fitting high-dimensional nonlinear models. Nhưng nếu simpler tools cho performance tương đương, chúng likely easier to fit/understand và potentially less fragile. Wherever possible, nên thử simple models rồi choose dựa trên performance/complexity tradeoff.

Thông thường deep learning attractive khi training sample size extremely large và model interpretability không phải high priority.

10.7 Fitting a Neural Network

Fitting neural networks tương đối complex; ở đây chỉ overview ngắn, ideas generalize tới much more complex networks. Readers thấy material challenging có thể skip. Good software trong lab cuối chapter cho phép fit neural-network models tương đối automated mà không phải worry về technical details của fitting procedure.

Bắt đầu với simple network Figure 10.1. Trong model (10.1), parameters là $\beta = (\beta_0, \beta_1, \dots, \beta_K)$ cùng $w_k = (w_{k0}, w_{k1}, \dots, w_{kp})$, $k = 1, \dots, K$. Với observations (x_i, y_i) , $i = 1, \dots, n$, có thể fit bằng nonlinear least squares:

$$\underset{\{w_k\}_1^K, \beta}{\text{minimize}} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2, \quad (10.23)$$

trong đó

$$f(x_i) = \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right). \quad (10.24)$$

Objective (10.23) nhìn simple, nhưng nested arrangement của parameters và symmetry của hidden units khiến minimization không straightforward. Problem nonconvex theo parameters, nên có multiple solutions. Figure

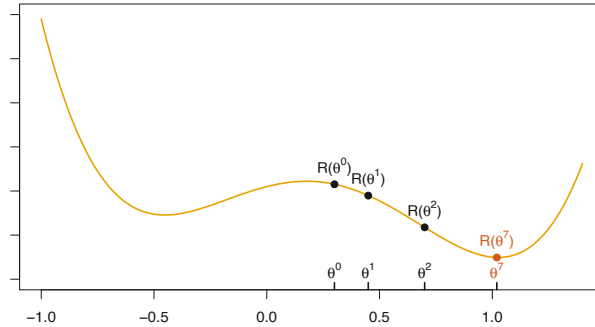
10.17 cho simple nonconvex function của one variable θ với two solutions: one local minimum và one global minimum. Đây còn là simplest neural network; more complex networks làm vấn đề compounded. Hai general strategies thường được dùng để overcome issues và protect from overfitting:

- *Slow Learning*: fit model bằng iterative gradient descent tương đối slow, rồi stop fitting process khi detect overfitting;
- *Regularization*: impose penalties lên parameters, thường lasso hoặc ridge như Section 6.2.

Represent tất cả parameters thành long vector θ . Objective (10.23) có thể viết

*local minimum;
global minimum;
gradient descent*

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2. \quad (10.25)$$



HÌNH 10.17. Illustration của gradient descent cho one-dimensional θ . Objective $R(\theta)$ không convex và có two minima: one tại $\theta = -0.46$ (local), one tại $\theta = 1.02$ (global). Bắt đầu từ θ^0 (typically randomly chosen), mỗi step đi downhill—against gradient—cho đến khi không thể xuống thêm. Ở đây gradient descent reach global minimum trong 7 steps.

Idea của gradient descent rất simple:

1. Start với guess θ^0 cho all parameters và set $t = 0$.
2. Iterate cho đến khi objective (10.25) không giảm nữa:
 - (a) find vector δ phản ánh small change trong θ , sao cho $\theta^{t+1} = \theta^t + \delta$ reduces objective, tức $R(\theta^{t+1}) < R(\theta^t)$;
 - (b) set $t \leftarrow t + 1$.

Có thể visualize việc đứng trên mountainous terrain và muốn đi xuống bottom qua series of steps. Miễn mỗi step downhill, cuối cùng ta tới một bottom. Example này lucky vì starting guess θ^0 dẫn tới global minimum; nói chung chỉ có thể hope đạt một good local minimum.

10.7.1 Backpropagation

Làm sao tìm direction để move θ nhằm decrease $R(\theta)$ trong (10.25)? Gradient của $R(\theta)$ tại current value $\theta = \theta^m$ là vector partial derivatives:

$$\nabla R(\theta^m) = \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^m}. \quad (10.26)$$

Subscript $\theta = \theta^m$ nghĩa là sau khi compute vector derivatives, ta evaluate tại current guess θ^m . Vector này cho direction trong θ -space mà $R(\theta)$ increases most rapidly. Gradient descent move một chút theo opposite direction:

$$\theta^{m+1} \leftarrow \theta^m - \rho \nabla R(\theta^m). \quad (10.27)$$

gradient

learning rate

Với learning rate ρ đủ small, step này decrease objective: $R(\theta^{m+1}) \leq R(\theta^m)$. Nếu gradient vector bằng zero, có thể ta đã reach a minimum.

Calculation (10.26) phức tạp đến đâu? Ở đây nó khá simple và vẫn simple với much more complex networks nhờ chain rule of differentiation. Vì $R(\theta) = \sum_{i=1}^n R_i(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$ là sum, gradient cũng là sum over n observations; chỉ cần examine one term:

$$R_i(\theta) = \frac{1}{2} \left[y_i - \beta_0 - \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right) \right]^2. \quad (10.28)$$

chain rule

Để simplify expressions, đặt $z_{ik} = w_{k0} + \sum_{j=1}^p w_{kj} x_{ij}$.

Derivative theo β_k :

$$\frac{\partial R_i(\theta)}{\partial \beta_k} = \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} = -(y_i - f_\theta(x_i)) g(z_{ik}). \quad (10.29)$$

Derivative theo w_{kj} :

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial w_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -(y_i - f_\theta(x_i)) \beta_k g'(z_{ik}) x_{ij}. \end{aligned} \quad (10.30)$$

Cả hai expressions chứa residual $y_i - f_\theta(x_i)$. Trong (10.29), một fraction của residual được attributed tới mỗi hidden unit theo $g(z_{ik})$; trong (10.30), similar attribution đi tới input j qua hidden unit k . Differentiation vì thế assign fraction của residual tới từng parameter thông qua chain rule—process gọi là *backpropagation*. Calculations straightforward nhưng cần careful bookkeeping.

backpropagation

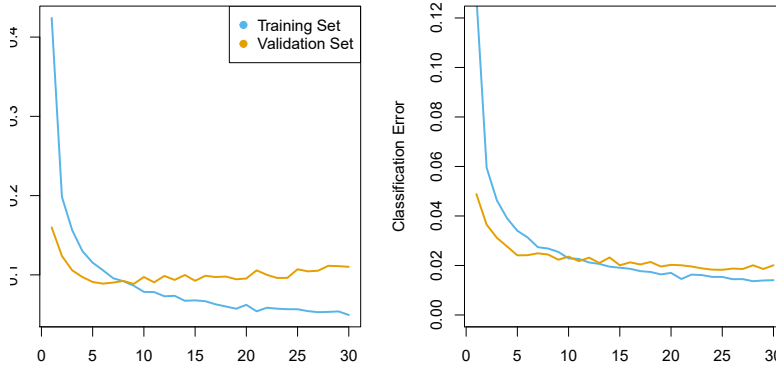
10.7.2 Regularization and Stochastic Gradient Descent

Gradient descent thường cần many steps để reach local minimum. Có nhiều approaches để accelerate process. Khi n large, thay vì sum (10.29)–(10.30) over all n observations, ta sample một small fraction hoặc *minibatch* mỗi lần compute gradient step. Process này gọi là *stochastic gradient descent* (SGD), state of the art cho learning deep neural networks. Good software

*minibatch;
stochastic gradient
descent*

giúp set up/fitting deep learning models và hide hầu hết technicalities khỏi user.

Quay lại multilayer network Figure 10.4 dùng cho digit recognition. Network có hơn 235,000 weights, khoảng four times number of training examples; regularization là essential.



HÌNH 10.18. Evolution của training và validation errors cho MNIST neural network Figure 10.4 theo training epochs. Objective refers to log-likelihood (10.14).

First row Table 10.1 dùng ridge regularization trên weights. Điều này được thực hiện bằng augment objective (10.14) với penalty term:

$$R(\theta; \lambda) = - \sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)) + \lambda \sum_j \theta_j^2. \quad (10.31)$$

Parameter λ thường preset small hoặc find bằng validation-set approach. Ta cũng có thể dùng different λ cho groups of weights ở different layers; trong example này W_1 và W_2 bị penalized, còn relatively few weights B của output layer không penalized. Lasso regularization cũng popular như additional form hoặc alternative to ridge.

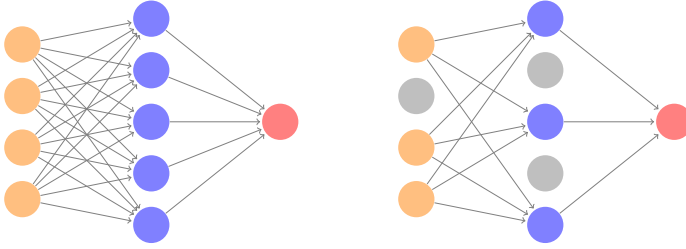
Figure 10.18 show metrics evolve trong training MNIST network. SGD tự nhiên enforce một form of approximately quadratic regularization.²¹ Mini-batch size ở đây là 128 observations per gradient update. Horizontal-axis term *epochs* counts number of times equivalent of full training set được processed. Với network này, 20% của 60,000 training observations được dùng làm validation set để determine when training should stop; thực tế 48,000 observations dùng train, nên có khoảng $48,000/128 \approx 375$ minibatch gradient updates per epoch. Validation objective bắt đầu increase vào khoảng 30 epochs, vì vậy *early stopping* cũng có thể act như additional regularization.

*epochs; early
stopping*

10.7.3 Dropout Learning

Second row Table 10.1 labeled *dropout*. Đây là relatively new và efficient form of regularization, similar in some respects to ridge. Inspired by random

²¹Property này và các properties khác của SGD cho deep learning là subject of much research trong machine learning literature tại thời điểm sách được viết.



HÌNH 10.19. *Dropout Learning. Left: fully connected network. Right: network với dropout trong input và hidden layer. Grey nodes được selected randomly và ignored trong một instance of training.*

forests (Section 8.2), idea là randomly remove fraction ϕ của units trong một layer khi fit model. Figure 10.19 minh họa. Việc remove được thực hiện separately mỗi khi process một training observation. Surviving units stand in for missing ones và weights được scale up factor $1/(1-\phi)$ để compensate. Điều này prevent nodes from becoming over-specialized và có thể xem như regularization. Trong practice dropout được thực hiện bằng randomly set activations của “dropped out” units về zero, vẫn giữ architecture intact.

dropout

10.7.4 Network Tuning

Network Figure 10.4 được xem là relatively straightforward nhưng vẫn cần nhiều choices ảnh hưởng performance:

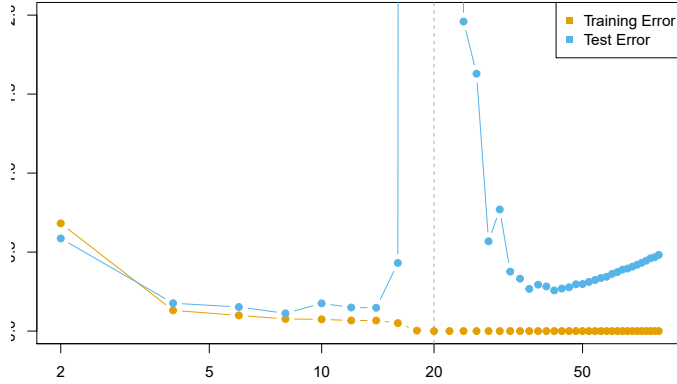
- number of hidden layers và units per layer. Modern thinking cho rằng units per hidden layer có thể large và overfitting controlled bằng regularization;
- regularization tuning parameters, gồm dropout rate ϕ và strength λ của lasso/ridge, thường set separately ở each layer;
- details của stochastic gradient descent: batch size, number of epochs và, nếu dùng, details của data augmentation (Section 10.3.4).

Những choices này có thể matter. Khi prepare MNIST example, authors đạt respectable 1.8% misclassification error sau some trial and error. Finer tuning/training của similar network có thể xuống dưới 1%, nhưng tinkering tedious và có thể cause overfitting nếu careless.

10.8 Interpolation and Double Descent

Trong suốt sách, ta repeatedly discuss bias-variance trade-off, first presented ở Section 2.2.2. Trade-off này chỉ ra statistical learning methods thường perform best về test-set error tại intermediate model complexity. Nếu plot “flexibility” trên x-axis và error trên y-axis, ta thường expect test error U-shaped trong khi training error decreases monotonically. Examples điển hình có ở Figure 2.9 (right panel) và Figure 2.17. Một implication là thường không nên *interpolate* training data—tức achieve zero training error—vì thường dẫn tới very high test error.

interpolate



HÌNH 10.20. Double descent phenomenon, illustrated bằng error plots cho one-dimensional natural-spline example. Horizontal axis là number of spline basis functions trên log scale. Training error hit zero khi degrees of freedom bằng sample size $n = 20$, “interpolation threshold”, rồi remain zero. Test error increases dramatically tại threshold nhưng sau đó descends again tới reasonable value trước khi cuối cùng increase trở lại.

Tuy vậy, trong certain settings, statistical learning method interpolate training data vẫn có thể perform well—hoặc ít nhất better than một slightly less complex model chưa hẳn interpolate. Phenomenon này gọi là *double descent*, shown trong Figure 10.20. Tên “double descent” vì test error có U-shape trước interpolation threshold rồi descend thêm lần nữa (ít nhất trong một khoảng) khi increasingly flexible model được fit.

Set-up cho Figure 10.20: simulate $n = 20$ observations từ

$$Y = \sin(X) + \varepsilon,$$

trong đó $X \sim U[-5, 5]$ và $\varepsilon \sim N(0, \sigma^2)$ với $\sigma = 0.3$. Sau đó fit natural spline như Section 7.4 với d degrees of freedom.²²

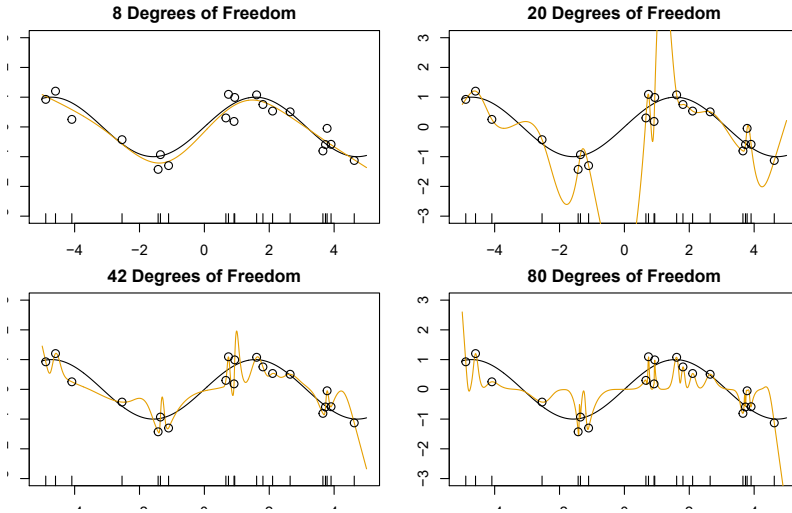
Recall rằng fit natural spline với d degrees of freedom tương đương fit least-squares regression của response lên set of d basis functions. Upper-left panel Figure 10.21 show data, true function $f(X)$ và $\hat{f}_8(X)$, fitted natural spline với $d = 8$.

Tiếp theo fit natural spline với $d = 20$. Vì $n = 20$, ta có $n = d$ và zero training error: training data được interpolated. Top-right panel Figure 10.21 cho thấy $\hat{f}_{20}(X)$ có wild excursions nên test error sẽ large.

Tiếp tục tăng d . Khi $d > 20$, least-squares regression của Y lên d basis functions không unique: có infinitely many coefficient estimates đạt zero error. Để choose, ta lấy solution có smallest sum of squared coefficients $\sum_{j=1}^d \hat{\beta}_j^2$, gọi là *minimum-norm solution*.

Hai lower panels Figure 10.21 show minimum-norm natural-spline fits với $d = 42$ và $d = 80$. Remarkably, $\hat{f}_{42}(X)$ much less wild than $\hat{f}_{20}(X)$ dù dùng more degrees of freedom; $\hat{f}_{80}(X)$ cũng không khác nhiều. Lý do: với $n = d = 20$ chỉ có one way để interpolate bằng 20 basis functions, và way đó tạo extreme fitted function. Nhưng với $d = 42$ hoặc 80 có infinitely

²²Điều này imply choice of d knots, ở đây chosen tại d equ-probability quantiles của training data. Khi $d > n$, quantiles được found by interpolation.



HÌNH 10.21. Fitted functions $\hat{f}_d(X)$ (orange), true function $f(X)$ (black), và 20 observed training data points. Mỗi panel dùng different d (degrees of freedom). Với $d \geq 20$, orange curves đều interpolate training points nên training error bằng zero.

many interpolating fits; smoothest one, tức minimum-norm solution, much less wild.

Figure 10.20 display training/test error của $\hat{f}_d(X)$ qua range of d . Training error drop to zero từ $d = 20$ trở đi, tức interpolation threshold. Test error U-shaped với $d \leq 20$, becomes extremely large quanh $d = 20$, rồi có second descent region khi $d > 20$. Trong example này signal-to-noise ratio $\text{Var}(f(X))/\sigma^2 = 5.9$, khá high, nên estimate interpolate data mà không wander quá xa giữa observed points likely perform well.

Figures 10.20–10.21 minh họa double descent trong simple one-dimensional natural-spline setting, nhưng phenomenon cũng có thể arise trong deep learning. Khi fit neural networks với huge number of parameters, đôi khi ta obtain good results với zero training error, đặc biệt ở high signal-to-noise problems như natural image recognition và language translation. Techniques fit neural networks, including SGD, naturally tend to select “smooth” interpolating model có good test performance trong những problems kiểu này.

Một số points cần emphasize:

- Double descent không contradict bias-variance trade-off. Right-hand double-descent curve trong Figure 10.20 xuất hiện vì x-axis là number of spline basis functions, không properly capture true “flexibility” của models interpolate training data. Nói cách khác, minimum-norm natural spline với $d = 42$ có lower variance than spline với $d = 20$.
- Most statistical learning methods trong sách không exhibit double descent. Regularization approaches thường không interpolate training data, nên double descent không occur; đây không phải drawback. Trong examples này, nếu natural splines được fit bằng ridge regression với appropriately-chosen penalty thay vì least squares, ta sẽ không

thấy double descent và thực tế test error còn better.

- Chapter 9 cho thấy maximal margin classifiers và SVMs có zero training error vẫn thường achieve very good test error, partly vì chúng seek smooth minimum-norm solutions. Điều này tương tự minimum-norm natural spline có good results với zero training error.
- Double-descent phenomenon được machine learning community dùng để explain practice dùng over-parametrized neural network (many layers, many hidden units) rồi fit tới zero training error. Tuy vậy, zero error không phải lúc nào optimal; advisability depend on signal-to-noise ratio. Có thể dùng ridge regularization để avoid overfitting neural network như (10.31); với suitable λ , model sẽ không interpolate training data nên không thấy double descent, nhưng vẫn có thể đạt very good test performance. Early stopping trong SGD cũng có thể act như regularization, prevent interpolation mà vẫn cho good test results.

Tóm lại, dù double descent đôi khi occur trong neural networks, thường ta không muốn rely on behavior này. Quan trọng hơn, bias-variance trade-off luôn hold; chỉ là test error as a function of “flexibility” có thể không exhibit U-shape nếu notion of flexibility trên x-axis được parametrized theo một cách không phù hợp.